



Hospital Appointment Management System – Review 1 Presentation Script



Member 1 – Introduction & Overview

Good morning everyone,

We're here to present our project – the **Hospital Appointment Management System**, developed using **Java, Spring Boot, MySQL, HTML, CSS, and JavaScript**.

This system aims to simplify and digitalize the healthcare appointment process by connecting **patients, doctors, and administrators** on a single web platform.

In traditional hospital setups, booking appointments involves long waiting times, manual errors, and lack of visibility into doctor availability.

Our project solves these problems through a **centralized digital system** where:

- Patients can search for doctors and book slots instantly
- Doctors can manage appointments and add prescriptions
- Admins can monitor hospital-wide analytics

The MVP we've built supports secure login, real-time booking, and dashboards for all three roles – showing how a hospital's workflow can be automated effectively.

(Transition)

Now, my teammate will explain our **problem statement and main objectives** in detail.



Member 2 – Problem Statement & Objectives

Thank you.

So, the **problem we addressed** was the inefficiency of traditional appointment booking systems. They rely on manual calls, causing confusion, double bookings, and lack of medical record tracking.

Our **main objectives** were:

1. To build a **role-based platform** for patients, doctors, and admins.
2. To implement **real-time doctor availability and slot booking**.

3. To enable **secure authentication** using JWT tokens.
4. To offer **appointment history and digital prescriptions** for patients.
5. And to provide **dashboard analytics** for doctors and admins.

For the **tech stack**, we chose:

- **Java 17 + Spring Boot 3.3.3** for backend logic and APIs.
- **Spring Security** for authentication and role-based access.
- **Spring Data JPA with Hibernate** for database management.
- **MySQL** for relational data handling.
- **Thymeleaf + HTML + CSS + Vanilla JS** for the frontend.

This combination allowed us to follow the **MVC + layered architecture**, making the project modular and scalable.

(Transition)

Next, my teammate will take you through the **system design and implementation** process.



Member 3 – System Design & Implementation

Thank you.

Our project follows a **layered architecture** with four main layers:

1. **Presentation Layer** – handles HTTP requests, controllers, and HTML templates.
2. **Service Layer** – contains business logic like booking validation and transaction handling.
3. **Repository Layer** – manages database operations through JPA repositories.
4. **Database Layer** – stores user, slot, and appointment data in MySQL.

For **authentication**, we used **JWT tokens** with Spring Security. When a user logs in, a token is generated and stored in both localStorage and cookies, ensuring session persistence for up to 30 days.

The **database schema** includes entities like:

- `User` (base class)
- `Patient`, `Doctor`, and `Admin` (derived)
- `Slot` and `Appointment` tables linked through relationships

We used **Single Table Inheritance** for user roles, reducing query complexity.

Key logic example:

When a patient books a slot, the system:

- Validates patient and doctor IDs
- Checks slot availability

- Marks the slot as booked
- Saves the appointment atomically using the `@Transactional` annotation

This prevents double-booking and ensures data consistency.

(Transition)

Now, my teammate will walk you through the **demo, results, and future enhancements**.

Member 4 – Demo, Results & Future Scope

Thank you.

In our demo, we showcase three dashboards:

- **Patient Dashboard** – lets patients search doctors by specialty, view slots (green for available, red for booked), and confirm appointments.
- **Doctor Dashboard** – displays daily appointments, allows marking them complete, and adding prescriptions.
- **Admin Dashboard** – shows analytics like appointment counts and system usage.

All core features – login, booking, cancellation, and prescription upload – are working smoothly in the MVP.

The system passed manual, API, and database tests with 100% functional coverage.

For **future scope**, we plan to:

- Add **email verification and password reset**
- Integrate **payment gateways (Razorpay/Stripe)**
- Implement **appointment reminders via SMS/email**
- Introduce **video consultations** and **advanced analytics**
- Containerize deployment using **Docker and CI/CD pipelines**

Overall, our system provides a scalable, secure, and user-friendly platform that bridges the gap between hospitals and patients through digital innovation.

(Conclusion)

Thank you for your attention – this concludes our presentation on the Hospital Appointment Management System.

We'll be happy to take your questions now.
